

EPFL CIVIL-127, Lab 4

March 10, 2026

This week, continue working in the same CIVIL-127 folder as previous weeks. Create a subfolder called lab4.

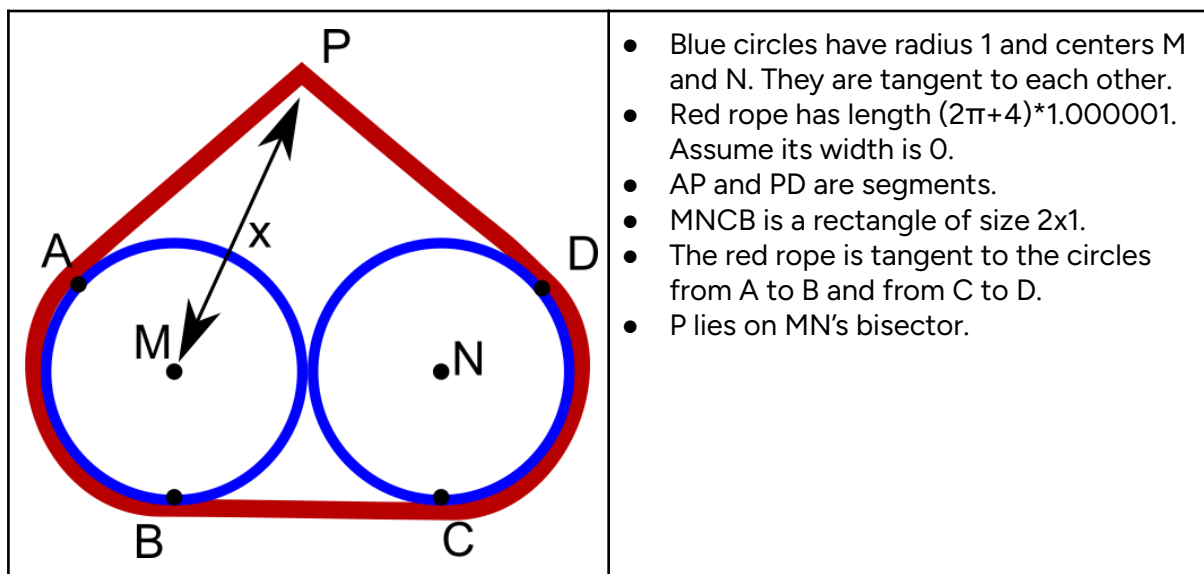
This week's lab continues to test your ability to implement logic. Can you apply the syntax and tools you have learned to solve problems?

To avoid struggling, I recommend debugging systematically. Be confident that each line of code you write is correct and doesn't have edge cases. If there are edge cases, document them and return to them later. Split large ideas and large pieces of code into smaller ones.

4.1

A rope is tied around two circles and pulled to remove any slack. Write a function to find an approximate value for x (length of PM) given the length of the red rope.

Note: most trigonometry operations in the `math` package use angles in radians, e.g. `math.acos` returns the arc cosine (inverse cosine) in radians.



If you are stuck and need a hint, highlight or copy-paste this:



Another hint:



4.2

Part 1

You are going to implement a battleship game generator.

Implement the following:

1. Create a `class`, call it `BattleShip`. In the `__init__` method, initialize a grid of size 13x9 that holds string values. You can use a single list, a list of lists, or a dictionary to represent the grid.
2. Add a method `randomize_grid`, which randomly places each ship in the grid. There are 5 ships, where each number indicates the ship's length:

```
ships = {  
    "a": (5, "carrier"),  
    "b": (4, "battleship"),  
    "c": (3, "cruiser"),  
    "d": (3, "submarine"),  
    "e": (2, "destroyer")  
}
```

You will want the grid to remember the ship which is there. I.e. you will want to store either the letters "a", "b", "c", ... or the names "carrier", "battleship", ... in the grid.

Ships can be placed either horizontally or vertically. Ships cannot overlap. One suggestion for structuring your code (other structures are equally good):

- In `randomize_grid`, loop over every ship type. For each ship of length `l`:
 - Randomly pick `x`, `y`, `d` in the ranges `[0-12]`, `[0-8]`, `[0-1]`; `x`, `y` are going to be the ship's top-left coordinates; `d` is going to be the direction.
 - Call `self.will_fit(...)` to first check if the ship is going to fit at this position
 - Then call `self.place(...)` or continue picking random numbers
- Implement a method `will_fit(self, length: int, x: int, y: int, d: int) -> bool`. This method should only check if the ship will fit in the grid and must not alter the grid.
- Implement a method `place(self, ship: str, length: int, x: int, y: int, d: int) -> None` which places the ship on the grid.

3. Implement a method `print_grid_with_ships` to print the grid with letters for the top row and numbers on the left side. It should look something like this:

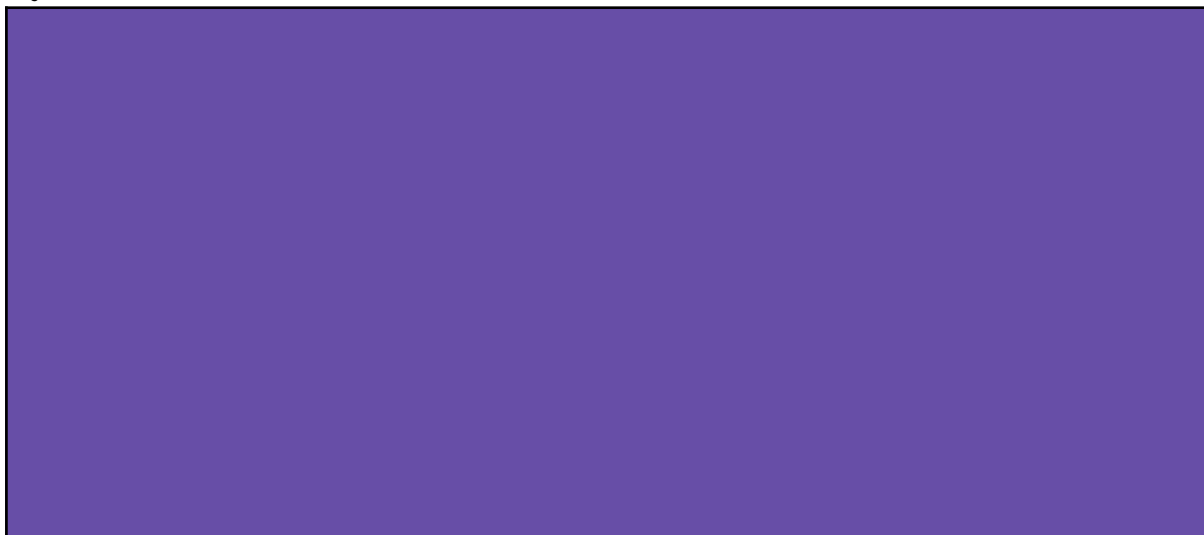
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	.	.	.	.	.	.	.	.	.	.	.	.	d
2	.	.	b	.	.	.	.	.	.	.	.	.	d
3	.	.	b	.	.	.	.	.	.	.	.	.	d
4	.	.	b	.	.	.	.	.	.	.	.	.	.
5	.	.	b	a	.	.	.	.	.	.	.	.	.
6	.	.	.	a	.	.	.	.	.	.	.	.	.
7	c	c	c	a	.	.	.	.	.	.	.	.	.
8	.	.	.	a	.	.	.	.	.	.	.	.	.
9	.	.	.	a	.	.	.	.	e	e	.	.	.

Run your code multiple times, make sure you always have the right number of ships and that they sometimes appear on the first row, last row, first column, last column, etc. Software is often buggy at edge cases!

Note: to convert a number `i` into a letter, you have several choices:

- `chr(i+ord("A"))`, uses the property that A-Z letters have consecutive Unicode code points
- `"ABCDEFGHIIJKLM"[i]`, which uses `i` to index in the string

If you are stuck and need a hint:



Part 2 (optional)

If you were quick to solve this lab, you can turn your code into a playable one-player game. Keep in mind that part 1 is about 50% of the work and implementing part 2 is going to take about as much code and time (hence it being optional):

- Create a data structure to track where the player has fired. Figure out how you are going to track whether a hit sinks a ship or not.
- Write a loop which asks the player to input coordinates in the form of <letter> + <number> (e.g. D5 or M8).
 - Respond by displaying an updated grid. This time, the grid shouldn't show the ships. Pick two characters to show miss vs hit (e.g. "~" and "#").
 - Display information about the shot: MISS/HIT <ship name>/SUNK <ship name>.
- When all the ships have been sunk, end the game.
- Celebrate! You just implemented a game from scratch, show it to your friends and family!